

Snort (스노트) 개념 정리

Snort 는 **실시간 트래픽 분석 및 패킷 로깅이 가능한 오픈소스 네트워크 침입 탐지 시스템(NIDS) 및 침입 방지 시스템(NIPS)**입니다. 1998년 Martin Roesch에 의해 개발되었으며, 현재는 Cisco가 소유하고 있습니다.

Snort 는 네트워크를 통과하는 트래픽을 실시간으로 모니터링하며, 사전에 정의된 '**룰(Rule)**' 또는 '**시그니처(Signature)**' 세트와 비교합니다. 만약 트래픽이 악성 코드, 포트 스캔, 버퍼 오버플로우 공격 등과 일치하는 패턴을 보이면, 이를 탐지하고 관리자에게 경고를 보내거나 해당 패킷을 차단할 수 있습니다.

1. 주요 특징 및 용도

주요 특징

- **오픈 소스:** 누구나 무료로 사용할 수 있으며, 전 세계 커뮤니티에 의해 룰셋(Rule Set)이 지속적으로 업데이트됩니다.
- **시그니처 기반 탐지:** 알려진 공격 패턴(시그니처)을 기반으로 트래픽을 검사하는 것이 핵심 기능입니다.
- **프로토콜 분석:** TCP, UDP, ICMP 등 다양한 프로토콜의 비정상적인 동작을 분석합니다.
- **다양한 동작 모드:** 단순 패킷 캡처부터 실시간 침입 방지까지 여러 모드로 동작할 수 있습니다.
- **확장성:** 사용자가 직접 커스텀 룰을 작성하여 새로운 위협에 대응할 수 있습니다.

주요 용도

- **침입 탐지 (IDS):** 네트워크에서 발생하는 악의적인 활동이나 정책 위반을 탐지하고 경고를 생성합니다.
- **침입 방지 (IPS):** 탐지된 악성 트래픽이 내부 네트워크로 도달하기 전에 실시간으로 차단합니다. (인라인 모드)
- **네트워크 포렌식:** 의심스러운 네트워크 활동을 패킷 단위로 로깅하여 사고 발생 후 원인을 분석하는 데 사용됩니다.
- **트래픽 모니터링:** `tcpdump` 처럼 네트워크 트래픽을 단순히 모니터링하는 용도로도 사용할 수 있습니다.

2. Snort의 3가지 주요 동작 모드

Snort 는 실행 시 어떤 옵션을 주느냐에 따라 3가지 모드 중 하나로 동작합니다.

1. 스니퍼 모드 (Sniffer Mode)

- `tcpdump` 와 거의 동일하게 동작합니다.
- 네트워크 패킷을 캡처하여 화면(콘솔)에 출력합니다.
- **사용 예:** `sudo snort -v` (단순 패킷 헤더 출력)
- **사용 예:** `sudo snort -vd` (패킷 헤더 + 데이터 링크 계층 출력)
- **사용 예:** `sudo snort -vde` (패킷 헤더 + 데이터 링크 + 페이로드(내용) 출력)

2. 패킷 로거 모드 (Packet Logger Mode)

- 캡처한 패킷을 화면에 출력하는 대신, 디스크에 `.pcap` 형식 등으로 저장합니다.
- 나중에 `tcpdump` 나 `Wireshark` 로 상세 분석이 필요할 때 사용합니다.

- **사용 예:** `sudo snort -l ./log -i eth0` (eth0 인터페이스의 모든 패킷을 ./log 디렉터리에 저장)

3. 침입 탐지/방지 (NIDS/NIPS) 모드

- `Snort` 의 핵심 기능입니다.
- 설정 파일(`snort.conf`)과 룰 파일(`local.rules` 등)을 읽어들이며, 네트워크 트래픽이 룰과 일치하는지 검사합니다.
- **NIDS (탐지) 모드:** 룰과 일치하는 패킷을 발견하면 경고(Alert)를 생성하고 로그에 기록합니다. (패킷을 차단하지는 않음)
- **NIPS (방지) 모드:** NIDS처럼 탐지할 뿐만 아니라, 해당 패킷을 ****차단(Drop)****하거나 ****거부(Reject)****합니다. (인라인(Inline) 구성 필요)
- **사용 예:** `sudo snort -c /etc/snort/snort.conf -i eth0 -A console`
 - `-c` : 사용할 설정 파일 지정
 - `-i` : 모니터링할 인터페이스 지정
 - `-A console` : 경고를 콘솔에 출력

3. Snort 룰(Rule)의 기본 구조

`Snort` 의 강력함은 '룰'에서 나옵니다. 룰은 "어떤 트래픽을" "어떻게 처리할지" 정의하는 규칙입니다. 룰은 ****헤더(Header)****와 **옵션(Option)** 두 부분으로 구성됩니다.

기본 구조: `[Action] [Protocol] [Src IP] [Src Port] -> [Dst IP] [Dst Port] ( [Options] )`

1. 헤더 (Header)

- `Action` : 룰과 일치할 때 수행할 동작 (예: `alert` , `drop` , `log` , `pass`)
- `Protocol` : 검사할 프로토콜 (예: `tcp` , `udp` , `icmp` , `ip`)
- `Src IP` : 출발지 IP (예: `any` , `192.168.0.1` , `$HOME_NET`)
- `Src Port` : 출발지 포트 (예: `any` , `80` , `1024:`)
- `->` : 트래픽 방향 (단방향)
- `Dst IP` : 목적지 IP
- `Dst Port` : 목적지 포트

헤더 예시: `alert tcp any any -> 192.168.1.0/24 80`

- **의미:** 모든 곳(any)에서 출발하여 192.168.1.0/24 네트워크의 80번 포트로 향하는 TCP 트래픽이 있다면 *경고(alert)*하라.

2. 옵션 (Option)

- 괄호 () 안에 작성하며, 세미콜론(;)으로 구분합니다.
- 패킷의 페이로드(실제 내용)를 검사하는 등 더 상세한 조건을 정의합니다.
- `msg` : 경고 시 출력할 메시지 (예: `msg:"WEB-ATTACK Detected";`)
- `content` : 패킷 내용(페이로드)에서 찾을 문자열 (예: `content:"/bin/bash";`)
- `sid` : 룰의 고유 식별 번호 (Snort ID)
- `rev` : 룰의 리비전(수정) 번호

전체 룰 예시:

```
alert tcp any any -> $HOME_NET 80 ( \
  msg:"WEB-ATTACK simple command execution"; \
  content:"/bin/bash"; \
  sid:1000001; \
  rev:1; \
)
```

- **의미:** 내부 네트워크(\$HOME_NET)의 80번 포트(웹 서버)로 들어오는 TCP 트래픽 중, 패킷 내용에 `/bin/bash` 라는 문자열이 포함되어 있다면 "WEB-ATTACK..." 메시지로 경고하라.

4. tcpdump vs. Snort vs. 방화벽(Firewall)

이 세 가지 도구는 자주 혼동되지만 역할이 명확히 다릅니다.

구분	tcpdump (패킷 분석기)	Snort (IDS/IPS)	방화벽 (Firewall)
주요 질문	"무슨 트래픽이 오고 가는가?"	"이 트래픽은 악의적인가?"	"이 트래픽은 허용되었는가?"
핵심 기능	캡처, 표시, 로깅	룰 기반 내용(페이로드) 분석, 탐지, 차단	정책 기반 접근 제어
판단 기준	없음 (모든 것을 보여줌)	시그니처 (룰 셋)	IP, Port, Protocol (정책)
비유	네트워크 현미경	네트워크 경비원	네트워크 문지기
주 사용처	문제 해결, 디버깅	보안 위협 탐지/차단	네트워크 경계 보호

요약:

- **tcpdump**는 모든 것을 **'보입니다'**.
- **방화벽**은 정책에 따라 트래픽을 **'허용/거부'**합니다. (주로 IP/Port 기반)
- **Snort**는 허용된 트래픽의 **'내용'**을 검사하여 악성 여부를 판단합니다.